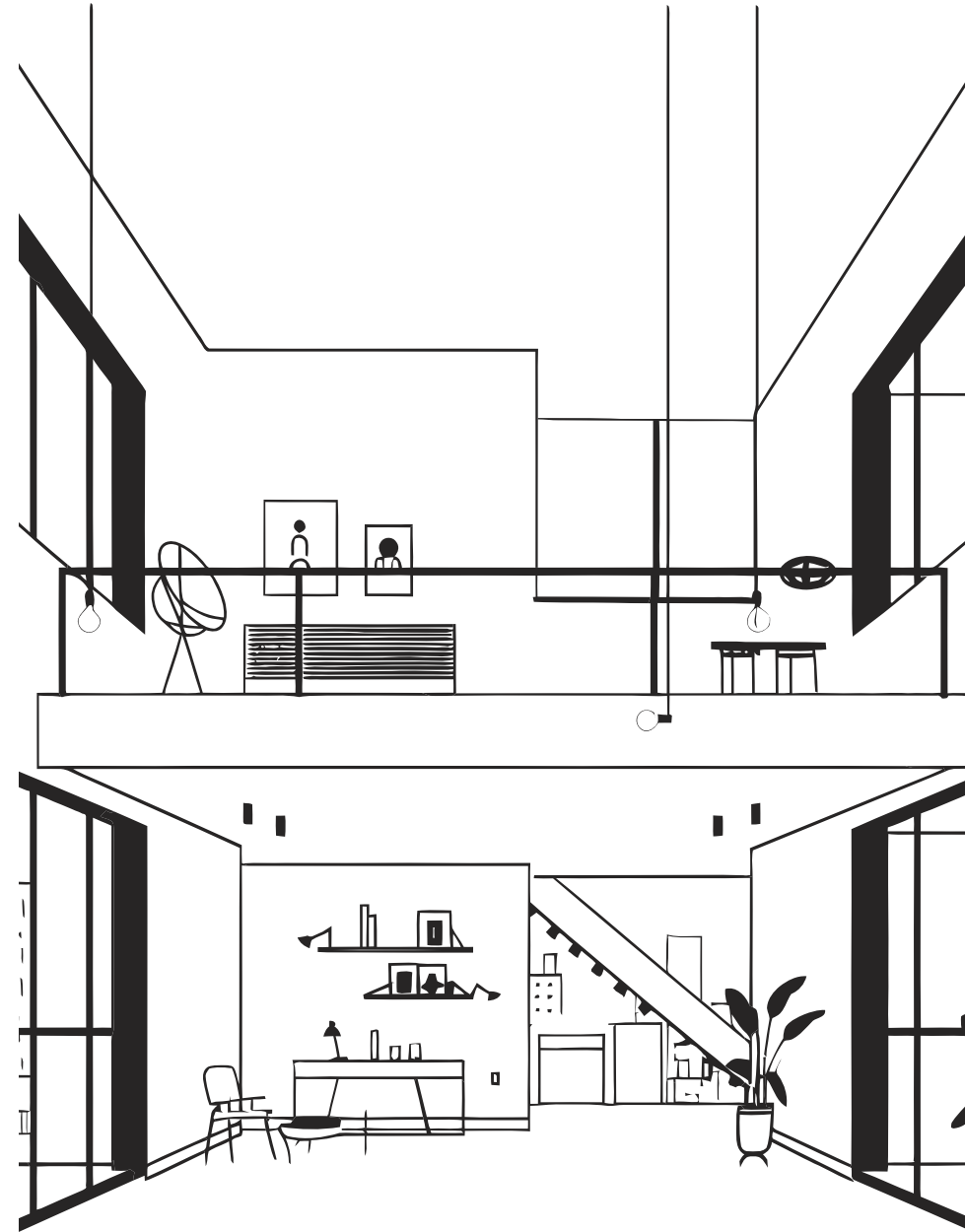


Teoria R3 · Framework, MVC i responsabilitats

De PHP base a un framework real. Convencions, estructura i on viu cada responsabilitat.

DAW · MÒDUL R3



De R2 a R3: no és copiar fitxers

R2 · PHP base

Fitxers solts

index.php, funcions globals, lògica barrejada amb HTML

Rutes manuals

URL directa al fitxer, sense enrutament centralitzat

Sense convenció

Cada projecte organitza les carpetes a la seua manera

R3 · Framework



Rutes declarades

Un lloc únic on es registren totes les URL



Capes separades

Controlador, dades i vista en llocs ben definits



Convencions comunes

L'equip sap on buscar cada peça sense preguntar



R3 és reconstruir amb criteri, no traslladar els fitxers de R2 a una carpeta nova.

Què és un framework?

Un framework és un **marc de treball amb convencions pactades**: carpetes, noms, cicle de vida de la petició i eines integrades. No cal inventar l'estructura; el framework ja l'ha decidit per tu.



Estructura predefinida

Saps on posar cada fitxer des del primer dia



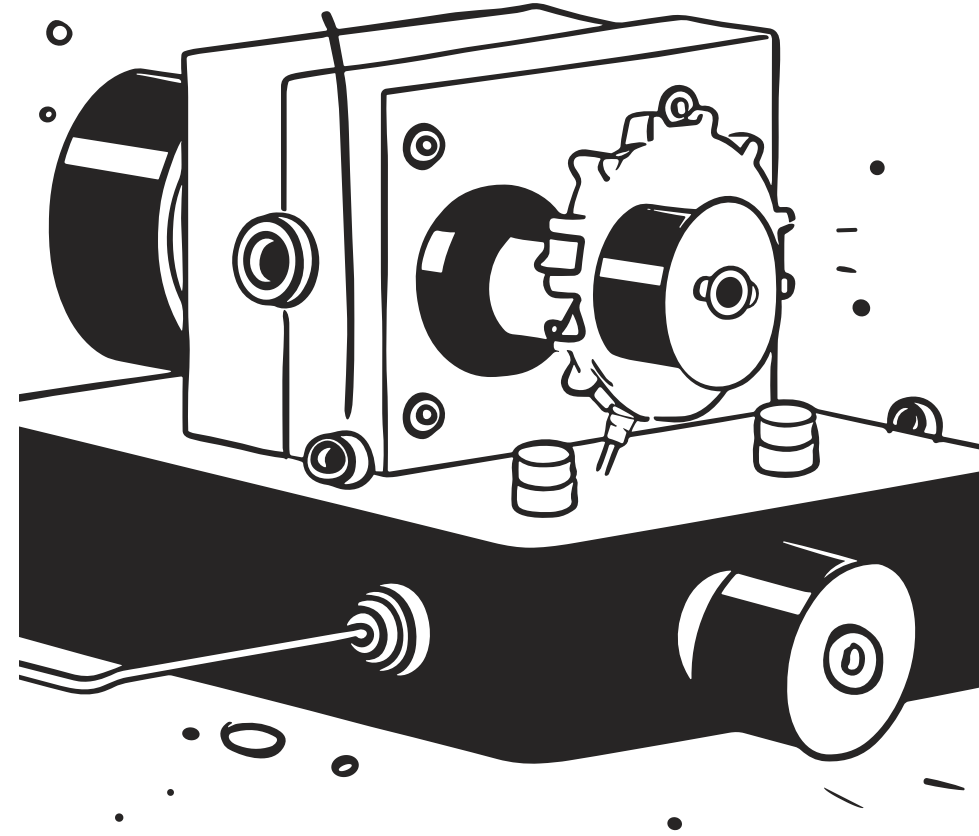
Eines integrades

Enrutament, validació, autenticació i més, ja incloses

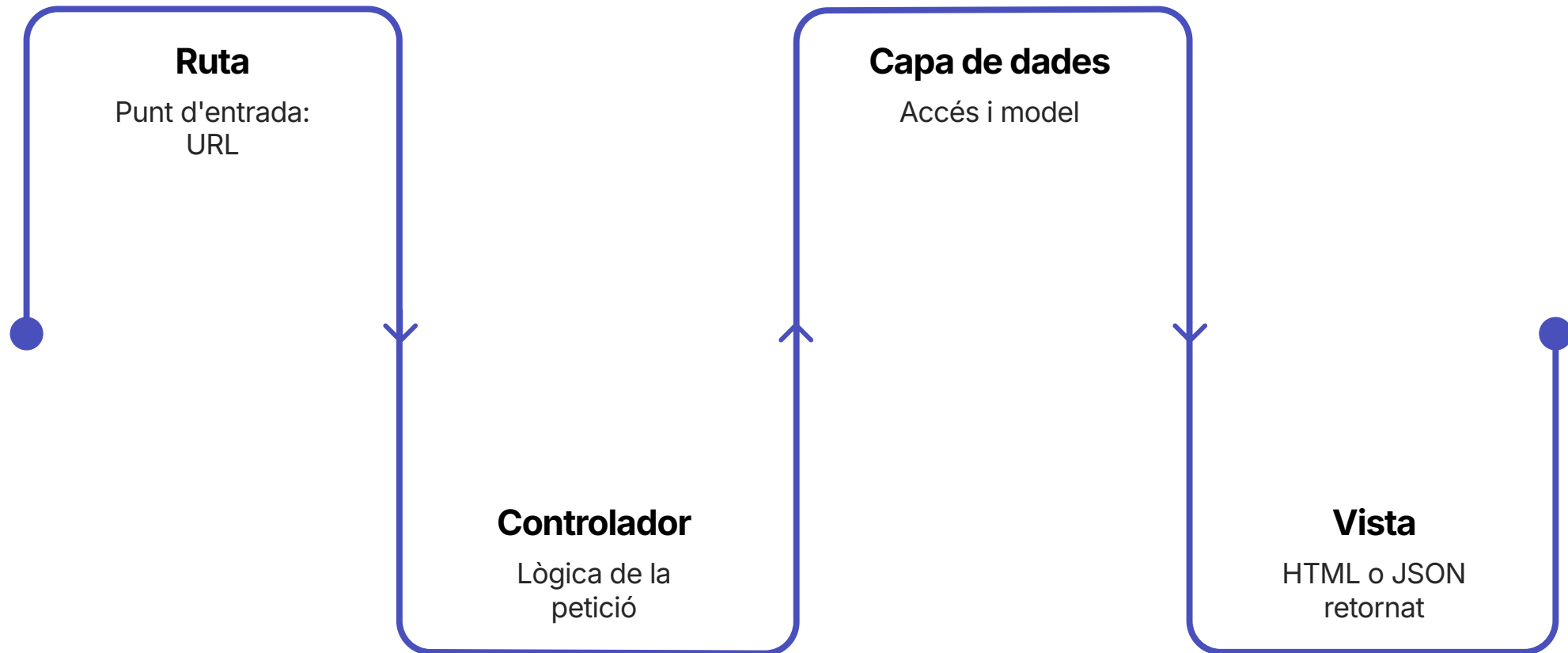


Ecosistema i comunitat

Documentació, paquets i bones pràctiques compartides

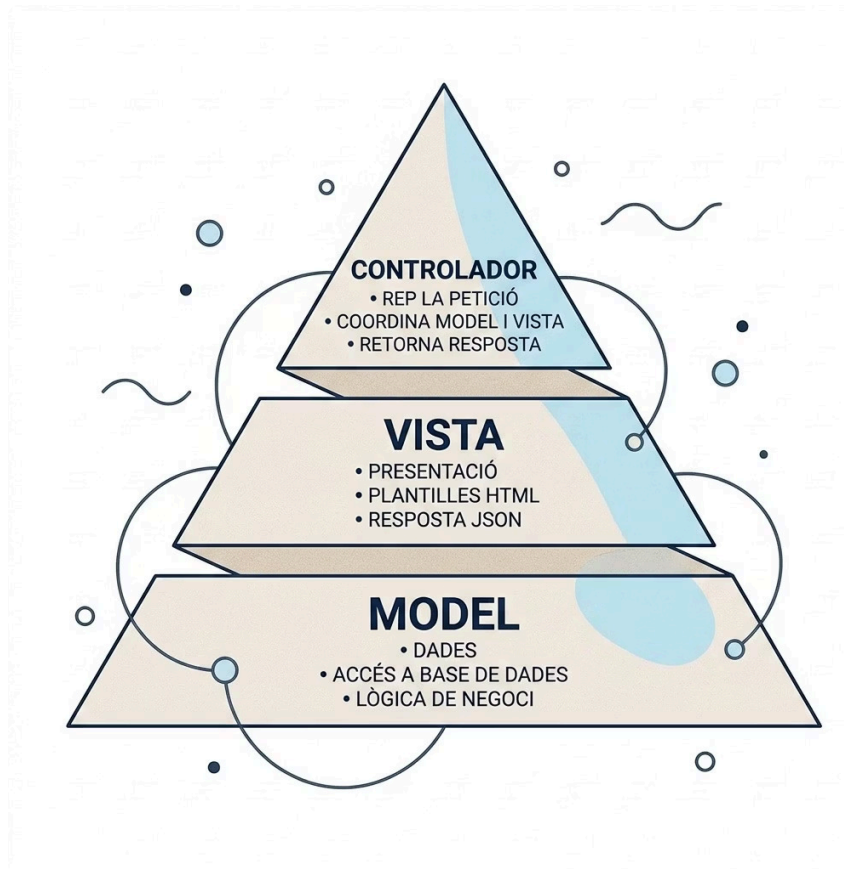


Les peces mínimes d'un framework



Aquestes quatre peces estan presents en tots els frameworks del mòdul. Entendre on viu cada responsabilitat és l'objectiu principal de R3.

MVC: on viu cada responsabilitat



Per què importa?

MVC no és teoria abstracta: és un **acord de localització**. Si saps quin problema tens, saps on mirar.

→ **Bug de dades?**

Mira el Model

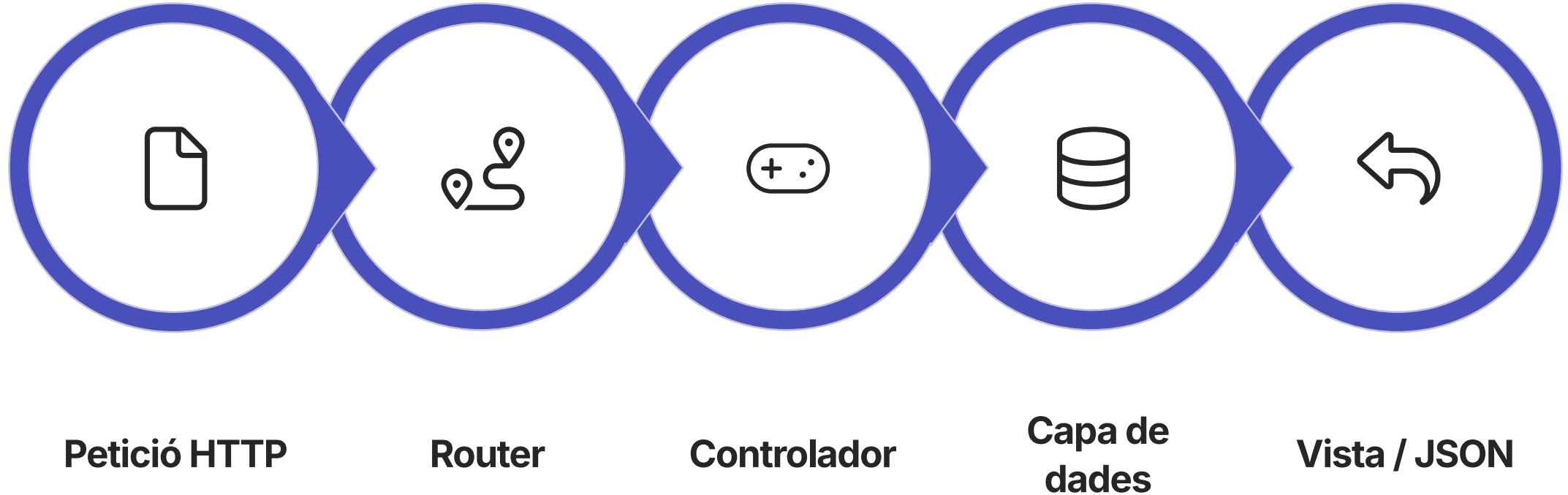
→ **Error de presentació?**

Mira la Vista

→ **Flux incorrecte?**

Mira el Controlador

El flux complet: petició → resposta



Aquest flux és el mateix en Laravel, Symfony i NestJS. Canvien els noms i la sintaxi, però **l'ordre de responsabilitats és idèntic**.

Equivalències entre frameworks

Necessitat	Laravel	Symfony	NestJS
Rutes	routes/web.php o api.php	Atributs o arxiu de config	Decoradors al controlador
Controladors	Controllers	Controllers	Controllers
Vista / Resposta	Blade o JSON	Twig o JSON	JSON o templates
Capa de dades	Eloquent	Doctrine	Prisma o TypeORM
Validació	Form Requests	Forms / Constraints	DTO + Pipes

 El framework triat ha de justificar-se amb avantatges concrets per al projecte, no per preferència personal.

Justificar el framework triat

No hi ha una opció correcta única. **Laravel, Symfony i NestJS** són vies vàlides si es compleixen les evidències comunes. La justificació ha de respondre:

Adequació al projecte

Per quina raó concreta encaixa amb el teu producte?

Avantatge tècnic

Quina eina o convenció et resol un problema real de R2?

Capacitat d'execució

Pots entregar 2 fluxos funcionals amb aquest stack?



Què aprofite de R2 i què refaig?

Aprofitar de R2

Lògica de negoci

Regles i càlculs que ja funcionen correctament

Fluxos identificats

Saps quines peticions ha de gestionar l'aplicació

Disseny de dades

L'estructura de taules o col·leccions que ja has validat

Refaire en R3

Enrutament

Passa a ser declarat en el sistema de rutes del framework

Estructura de fitxers

Segueix les carpetes i convencions del framework escollit

Accés a dades

S'integra a través de l'ORM o capa de dades del framework

 El flux de R2 serveix de guia per decidir quins 2 fluxos entren primer en R3.

Errors habituals en la transició

Copiar fitxers de R2 directament

El framework no reconeixerà l'estructura. La lògica barrejada trenca el cicle de vida de la petició.

No entendre les carpetes del framework

Posar controladors a la carpeta equivocada o inventar-se una estructura paral·lela genera confusió immediata.

Trencar el flux ruta → controlador

Cridar directament a fitxers o funcions sense passar per l'enrutament elimina totes les convencions del framework.



Què entra a R3M1 i què no

✓ Entra a R3M1

- 2 fluxos funcionals complets end-to-end
- Estructura de projecte seguint el framework triat
- Rutes declarades al sistema d'enrutament
- Controladors separats de la capa de dades
- Vistes o respostes en la capa corresponent
- Justificació escrita de l'stack triat

✗ No entra a R3M1

- Migració completa de tota l'aplicació R2
- ORM, migracions o seeders en profunditat
- Endpoints d'API REST (pertany a R4)
- Autenticació avançada o gestió de rols
- Optimitzacions de rendiment o caching

❗ L'objectiu és construir una base real i sòlida, no un producte complet. Qualitat sobre quantitat.

Checklist R3M1

1

Framework instal·lat

Projecte nou creat seguint la documentació oficial

2

Stack justificat

Document o apartat explicant per què aquest framework per a aquest projecte

3

2 fluxos complets

Ruta → Controlador → Dades → Vista/Resposta funcionant

4

Estructura respectada

Carpetes i fitxers seguint les convencions del framework

- ☐ Framework instal·lat i projecte inicialitzat
- ☐ Justificació de l'stack redactada
- ☐ Primer flux end-to-end funcionant
- ☐ Segon flux end-to-end funcionant

Preguntes de comprovació

Respon aquestes preguntes per verificar que has entès la transició a R3.

1

Quin flux de R2 continuarà viu en R3?

Identifica les peticions principals del teu projecte que han de funcionar des del primer dia.

2

Què canvia en passar a framework?

Enumera almenys tres diferències concretes entre PHP base i el framework triat.

3

On viu cada responsabilitat?

Indica la carpeta o fitxer on posaries una ruta, un controlador, una vista i una regla de validació.

4

Com justificaries el teu framework?

Argumenta l'elecció amb avantatges concrets per al teu projecte, no per preferències generals.